འབྲུག་རྒྱལ་འཛིན་གཞུག་ལག་སློབ་སྡེ།།

College of Science and Technology
Rinchending: Bhutan

WEB101
Web Application Fundamentals
(SS2026)

Practical 6 Report

Submitted By;
Name: Khamsum Yoezer
Student No.: 02250390
Programme: BE 1SWE
Date: 17/5/2026

RUB Wheel of Academic Law: Academic Dishonesty

Section H2 of the Royal University of Bhutan's *Wheel of Academic Law* provides the following definition of academic dishonesty:

Academic dishonesty may be defined as any attempt by a student to gain an unfair advantage in any assessment. It may be demonstrated by one of the following:

1. **Collusion:** the representation of a piece of unauthorised group work as the work of a single candidate.
2. **Commissioning:** submitting an assignment done by another person as the student's own work.
3. **Duplication**: the inclusion in coursework of material identical or substantially similar to material which has already been submitted for any other assessment within the University.
4. **False declaration**: making a false declaration to receive special consideration by an Examination Board or to obtain extensions to deadlines or exemption from work.
5. **Falsification of data**: presentation of data in laboratory reports, projects, etc., based on work purported to have been carried out by the student, which has been invented, altered or copied by the student.
6. **Plagiarism**: the unacknowledged use of another's work as if it were one's own.

Examples are:

- verbatim copying of another's work without acknowledgement.
- paraphrasing of another's work by simply changing a few words or altering the order of presentation, without acknowledgement.
- ideas or intellectual data in any form presented as one's own without acknowledging the source(s).
- making significant use of unattributed digital images such as graphs, tables, photographs, etc., taken from test books, articles, films, plays, handouts, the internet, or any other source, whether published or unpublished.
- submission of a piece of work which has previously been assessed for a different award or module or at a different institution as if it were new work.
- use of any material without prior permission of copyright from the appropriate authority or owner of the materials used".

Aim

The aim of this practical is to develop a Todo List application using React and Zustand for state management. The practical also aims to demonstrate how Zustand simplifies global state management and persistence in React applications.

Theory

State Management

State management refers to the process of storing and managing data within an application. In React applications, state is used to control dynamic data such as user input, lists, and UI updates.

Traditional React applications often use:

- Prop drilling
- Context API
- Redux

However, these approaches can become complex in large applications.

Zustand

Zustand is a lightweight state management library for React applications.

Features of Zustand

- Simple and minimal syntax
- No boilerplate code
- Global state management
- Fast performance
- Easy integration with React
- Built-in middleware support

Zustand uses a central store to manage application state.

Todo List Application

A Todo List application allows users to:

- Add tasks
- Mark tasks as completed
- Delete tasks
- Clear completed tasks

This practical demonstrates how Zustand can manage these operations efficiently.

Persistence with localStorage

Persistence means saving application data even after the browser is refreshed or closed.

In this project:

- Zustand persist middleware stores todos in localStorage
- Data remains available after refreshing the page

Implementation Steps

Step 1: Create React Project

A new React project was created using Vite.

Commands Used

npx create-vite@latest todo-zustand
cd todo-zustand
npm install

Step 2: Install Zustand

The Zustand package was installed.

npm install zustand

Purpose

This package provides global state management functionality.

Step 3: Create Zustand Store

A Zustand store was created to manage todo data.

Features Added

- Store todo items
- Add todos
- Toggle completed status
- Delete todos
- Clear completed todos

The store also included persistence using middleware.

```
import { create } from "zustand";
import { persist } from "zustand/middleware";

const useTodoStore = create(
  persist(
    (set) => ({
      todos: [],

      addTodo: (text) =>
        set((state) => ({
          todos: [
            ...state.todos,
            {
              id: Date.now(),
              text,
              completed: false,
            },
          ],
        })),

      toggleTodo: (id) =>
        set((state) => ({
          todos: state.todos.map((todo) =>
            todo.id === id
              ? { ...todo, completed: !todo.completed }
              : todo
          ),
        })),

      removeTodo: (id) =>
        set((state) => ({
          todos: state.todos.filter((todo) => todo.id !== id),
        })),

      clearCompleted: () =>
        set((state) => ({
          todos: state.todos.filter((todo) => !todo.completed),
        })),
    }),
    {
      name: "todo-storage",
    }
  )
);

export default useTodoStore;
```

Step 4: Create Todo Input Functionality

An input field and add button were implemented.

Features

- Accept user task input
- Add tasks to the store
- Clear input after submission

```jsx
import { useState } from "react";
import useTodoStore from "../store/todoStore";

Tabnine | Edit | Test | Explain | Document
function TodoInput() {
  const [text, setText] = useState("");
  const addTodo = useTodoStore((state) => state.addTodo);

  const handleAdd = () => {
    if (text.trim() === "") return;

    addTodo(text);
    setText("");
  };

  return (
    <div>
      <input
        type="text"
        placeholder="Enter todo..."
        value={text}
        onChange={(e) => setText(e.target.value)}
      />

      <button onClick={handleAdd}>Add</button>
    </div>
  );
}

export default TodoInput;
```

Step 5: Display Todo Items

Todo items were displayed dynamically using array mapping.

Features

- Show all todos
- Display completed tasks with line-through styling
- Update UI automatically when state changes

```jsx
import useTodoStore from "../store/todoStore";

Tabnine | Edit | Test | Explain | Document
function TodoItem({ todo }) {
  const toggleTodo = useTodoStore((state) => state.toggleTodo);
  const removeTodo = useTodoStore((state) => state.removeTodo);

  return (
    <li>
      <span
        onClick={() => toggleTodo(todo.id)}
        style={{
          textDecoration: todo.completed ? "line-through" : "none",
          cursor: "pointer",
        }}
      >
        {todo.text}
      </span>

      <button onClick={() => removeTodo(todo.id)}>
        Delete
      </button>
    </li>
  );
}

export default TodoItem;
```

Step 6: Add Toggle Functionality

Users can click on a todo item to mark it as completed or incomplete.

Result

- Completed tasks show line-through text decoration
- State updates instantly

```
  onClick={() => toggleTodo(todo.id)}
  style={{
    textDecoration: todo.completed ? "line-through" : "none",
    cursor: "pointer",
  }}
>
  {todo.text}
</span>
```

Step 7: Add Delete Functionality

Each todo item includes a delete button.

Purpose

- Remove unwanted tasks from the list

```
      <button onClick={() => removeTodo(todo.id)}>
        Delete
      </button>
    </li>
  );
```

Step 8: Add Clear Completed Functionality

A button was implemented to remove all completed tasks.

Benefit

- Keeps the todo list organized
- Removes finished tasks quickly

```jsx
import useTodoStore from "../store/todoStore";
import TodoItem from "./TodoItem";

Tabnine | Edit | Test | Explain | Document
function TodoList() {
  const todos = useTodoStore((state) => state.todos);
  const clearCompleted = useTodoStore(
    (state) => state.clearCompleted
  );

  return (
    <div>
      <ul>
        {todos.map((todo) => (
          <TodoItem key={todo.id} todo={todo} />
        ))}
      </ul>

      <button onClick={clearCompleted}>
        Clear Completed
      </button>
    </div>
  );
}

export default TodoList;
```

Step 9: Add Persistence

The Zustand persist middleware was used.

Purpose

- Save todos in browser localStorage
- Automatically reload data after page refresh

```
import { useState } from "react";
import { create } from "zustand";
import { persist } from "zustand/middleware";

// Zustand Store
const useTodoStore = create(
  persist(
    (set) => ({
      todos: [],

      addTodo: (text) =>
        set((state) => ({
          todos: [
            ...state.todos,
            {
              id: Date.now(),
              text,
              completed: false,
            },
          ],
        })),

      toggleTodo: (id) =>
        set((state) => ({
          todos: state.todos.map((todo) =>
            todo.id === id
              ? { ...todo, completed: !todo.completed }
              : todo
          ),
        })),
```

```
      removeTodo: (id) =>
        set((state) => ({
          todos: state.todos.filter((todo) => todo.id !== id),
        })),

      clearCompleted: () =>
        set((state) => ({
          todos: state.todos.filter((todo) => !todo.completed),
        })),
    }),
    {
      name: "todo-storage",
    }
  )
);

Tabnine | Edit | Test | Explain | Document
function App() {
  const [text, setText] = useState("");

  const todos = useTodoStore((state) => state.todos);
  const addTodo = useTodoStore((state) => state.addTodo);
  const toggleTodo = useTodoStore((state) => state.toggleTodo);
  const removeTodo = useTodoStore((state) => state.removeTodo);
  const clearCompleted = useTodoStore(
    (state) => state.clearCompleted
  );

  const handleAdd = () => {
    if (text.trim() === "") return;
```

Output

After successful implementation:

- Users can add new todos
- Todos can be marked completed
- Todos can be deleted
- Completed todos can be cleared

- Data remains saved after refreshing the browser

The application provides a simple and responsive task management interface.

Conclusion

In this practical, a Todo List application was successfully developed using React and Zustand. Zustand simplified state management by eliminating complex boilerplate code and allowing direct access to global state. Persistence was also implemented using localStorage, ensuring that user data remains available after refreshing the application. Overall, the practical demonstrated how Zustand provides an efficient and lightweight solution for state management in React applications.

References

1. Zustand Documentation
   Poimandres. (2026). *Zustand documentation*. Retrieved May 15, 2026, from https://zustand-demo.pmnd.rs/

2. React Documentation
   Meta. (2026). *React documentation*. Retrieved May 15, 2026, from https://react.dev/

3. Vite Documentation
   Vite. (2026). *Vite guide*. Retrieved May 15, 2026, from https://vitejs.dev/guide/

4. MDN localStorage Documentation
   Mozilla Developer Network. (2026). *Window: localStorage property*. Retrieved May 15, 2026, from https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

5. JavaScript Array map() Documentation
   Mozilla Developer Network. (2026). *Array.prototype.map()*. Retrieved May 15, 2026, from https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/map